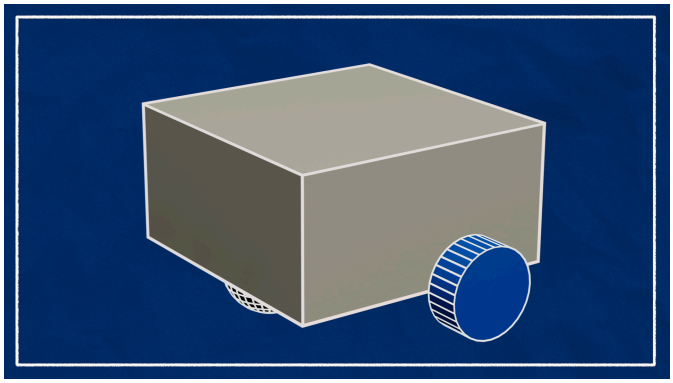
URDF Design

In this tutorial we're looking at the concept design of the robot, understanding the differential drive approach and creating a rough URDF (structural description). The post grew a little too long to include the initial simulation, so we'll cover that in the next one.

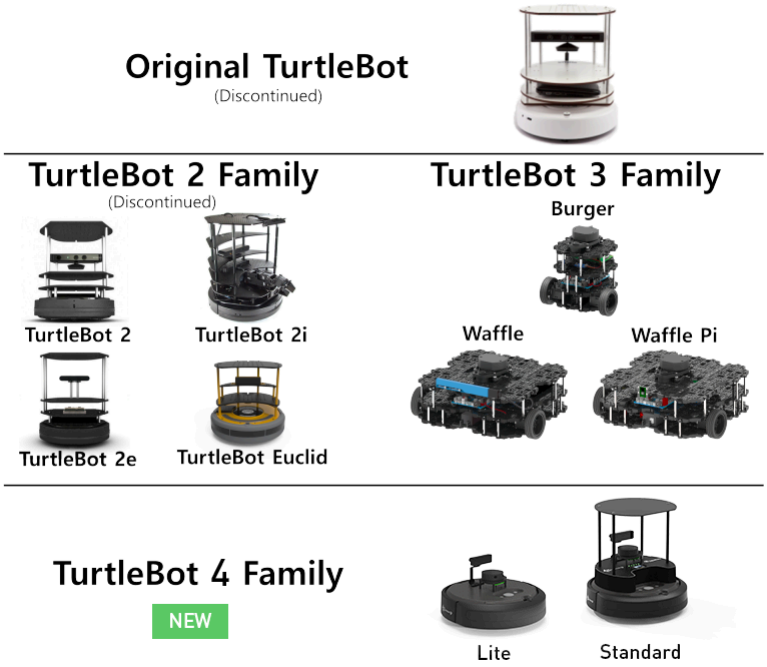
Differential Drive Robots

Differential-drive control

This robot is a *differential-drive* robot, which means the robot has two driven wheels, one on the left and one on the right. These two wheels control all motion, and any other wheels are just there to keep it stable and can spin freely in all direction (these are called caster wheels). This arrangement is popular for robots as it is simple to understand, build, and control, and is also quite nimble since it can turn on the spot (unlike, say, a car that requires space for a U-turn or three-point-turn).



Differential-drive robots can come in a variety of shapes and sizes. For today, I'm going to pretend our robot is a small box, with the driven wheels near the rear and a caster near the front. You may like to design yours a little differently - here are some examples of the various iterations of the "TurtleBot" design, a popular ROS robot for education and research.



Mobile Robots in ROS

Since our robot is a mobile robot we'll be following some of the conventions set out in in the ROS REPs (essentially the standards):

- The main coordinate frame for the robot will be called `base_link` (REP 105 - Coordinate Frames for Mobile Platforms)
- The orientation of this coordinate frame will be X-forward, Y-left, Z-up (REP 103 - Standard Units of Measure and Coordinate Conventions)

Working With URDF Files

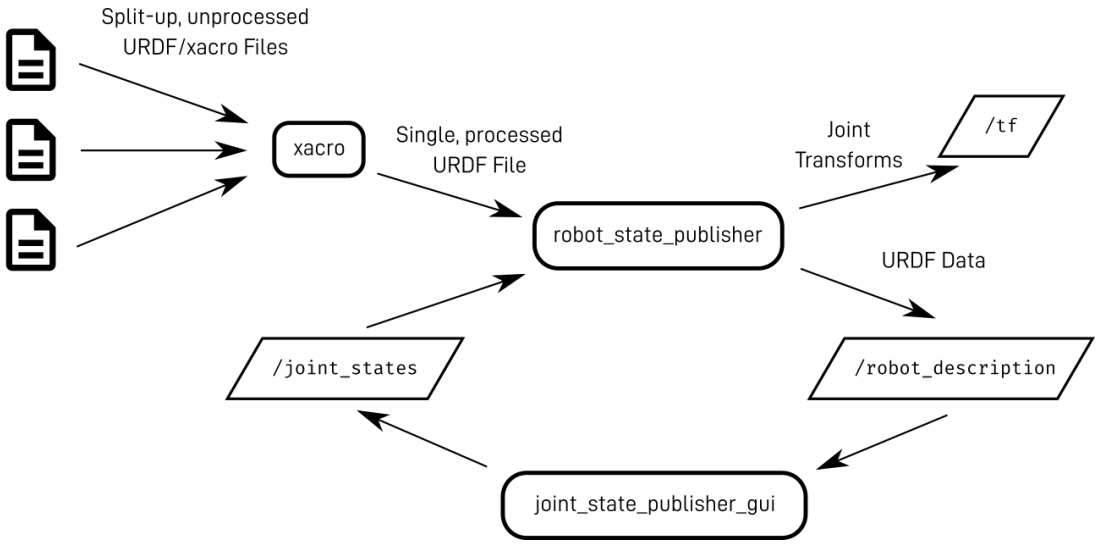
Let's start by covering some basics of how to work with URDF files.

Quick Recap

First, we should recap how to publish a robot description using URDFs. We start off with a bunch of files in the URDF format, that together describe our robot.

These files are processed by a tool called `xacro` which combines them into a single, complete URDF. This is passed to `robot_state_publisher` which makes the data available on the `/robot_description` topic, and also broadcasts the appropriate transforms.

If there are any joints that move, `robot_state_publisher` will expect to see the input values published on the `/joint_states` topic, and while doing our initial tests we can use the `joint_state_publisher_gui` to fake those values.



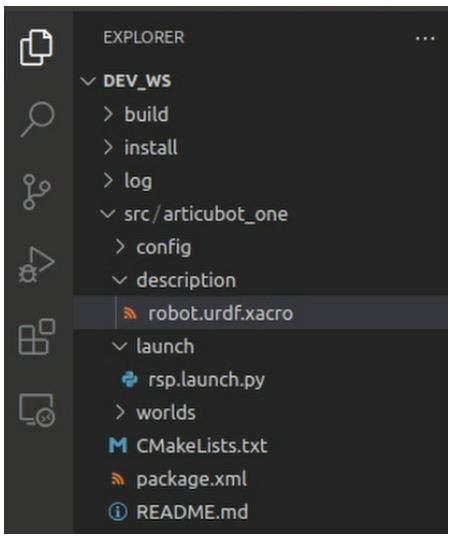
To make life easier, we typically wrap this up into a launch file.

Before we move on, we also want to make sure that `xacro` and `joint_state_publisher_gui` are installed (if they aren't already):

```
sudo apt install ros-foxy-xacro ros-foxy-joint-state-publisher-gui
```

Launching and Visualising

If you made a copy of the package from the GitHub template repo, you should already have a launch file (`launch/rsp.launch.py`) and a `description` directory containing a very basic URDF, `robot.urdf.xacro`.

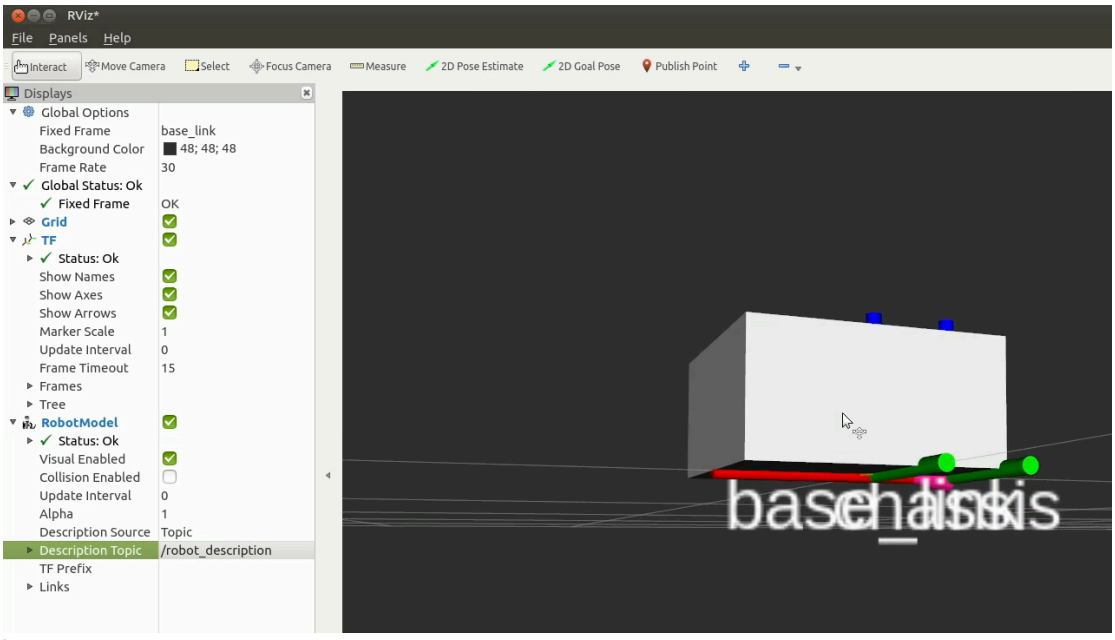


Try launching that file with `ros2 launch my_bot rsp.launch.py` (where `my_bot` is whatever you called your package, mine is `articubot_one`). You should see the output displayed below:

```
dev@dev-PC:~$ cd dev_ws/
dev@dev-PC:~/dev_ws$ source install/setup.bash
dev@dev-PC:~/dev_ws$ ros2 launch articubot_one rsp.launch.py
[INFO] [launch]: All log files can be found below /home/dev/.ros/log/2022-02-23-18-03-29-135260-dev-PC-2
1383
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [21409]
[robot_state_publisher-1] Parsing robot urdf xml string.
[robot_state_publisher-1] [INFO] [1645599809.270515255] [robot_state_publisher]: got segment base_link
```

Because this URDF file only has a single link (and it's empty), there won't be anything to display in RViz yet. Once we have added a couple of links and something to display (at the end of the *Chassis* section below), we'll be able to open RViz and do the following:

- Set the fixed frame to `base_link`
- Add a TF display (and enable showing names)
- Add a RobotModel display (setting the topic to `/robot_description`)



> Note: If you'd like to save your RViz setup for future use, you can do so in the file menu. Then to use it, when running RViz you can use the `-d` argument and the path to the `.rviz` file. E.g. `rviz2 -d ~/path/to/my/config/file.rviz`.

Tricks when making changes to URDFs

Before we start creating our URDF files there are a couple of things to be aware of when saving our changes.

- If you build your workspace with `colcon build --symlink-install` then you don't need to rebuild every time you update the URDF, it will be automatic EXCEPT whenever you add a new file. That file needs to get built/copied once, and after that will be kept updated.
- You'll need to quit and relaunch `robot_state_publisher` each time you make a change.
- RViz sometimes won't pick up all your changes straight away. In this case you'll need to hit the "Reset" button in the bottom corner. If it's still not updating, try ticking and unticking the display items, or closing and reopening the program.

Creating our URDF

Now it's time to create a URDF for our robot!

Splitting up our files

For this robot, instead of keeping all our configuration in a single URDF file, we'll be splitting it up into multiple files and *including* them in a main file. In this post we'll be focusing on a file that contains most of the core structure of the robot, and later on we will create more files for our sensors etc. and we can include them all in here to keep things organised.

To begin with, open up `robot.urdf.xacro` from the template and delete the `base_link` that is currently there. Replace it with the line `<xacro:include filename="robot_core.xacro" />`, so that your file looks like this:

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"  name="robot">

    <xacro:include filename="robot_core.xacro" />

</robot>
```

If we try to relaunch `robot_state_publisher` now, it will fail, because it is trying to include a file called `robot_core.xacro` that doesn't exist - so we better make it!

Creating the visual structure

We're about to start creating the visual structure for our robot. Fair warning, we'll start to get into a bit of maths and geometry and spatial stuff here, which can be a bit confusing if you're not used to it. Try to follow along, but if you get confused you should be able to copy-and-paste and muddle your way to the end, and hopefully things will make more sense in hindsight.

Making the Core File

In the `description` directory, create a new file called `robot_core.xacro`. To ensure it's a valid URDF, we'll start by copying the XML declaration and the `robot` tags (these are the same as in the previous file but without the `name` parameter). All of our links and joints will be going inside the robot tag, and we'll step through them one by one.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

    ... all our links and joints will go in here ...

</robot>
```

Colours

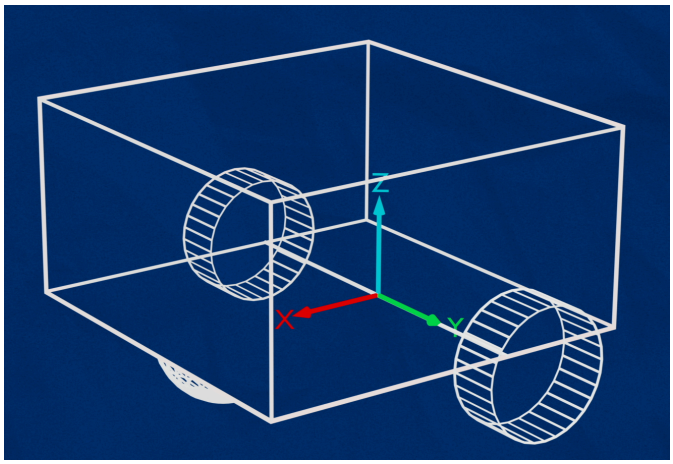
As mentioned in the URDF tutorial, we have the option to declare some materials (colours) up-front so that we can use them later. The colours are specified as float RGB triplets with an alpha channel. If that's meaningless to you, don't worry too much. You can start with these colours and if you want more, use this colour picker and copy the values from the "RGB 0.0-1.0 float" section.

You can either put these into a separate file (e.g. `colours.xacro`, remember to add your robot tags and include the new file), or at the top of this file just inside the robot tag.

```
<material name="white">
    <color rgba="1 1 1 1"/>
</material>

<material name="orange">
    <color rgba="1 0.3 0.1 1"/>
</material>

<material name="blue">
    <color rgba="0.2 0.2 1 1"/>
</material>

<material name="black">
    <color rgba="0 0 0 1"/>
</material>
```

Base Link

As mentioned earlier, it is standard in ROS for the main "origin" link in a mobile robot to be called `base_link`. You might think the natural place for the base link is in the centre of the chassis somewhere - and this wouldn't necessarily be wrong - but for a differential drive robot it is simplest to treat the centre of the two drive wheels as the origin, since the rotation will be centred around this point. The rest of the robot can then be described from there. So we start with an empty link called `base_link`.

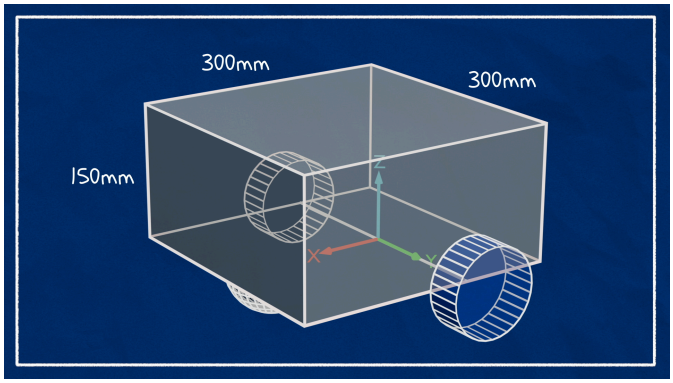



```
<link name="base_link">
</link>
```

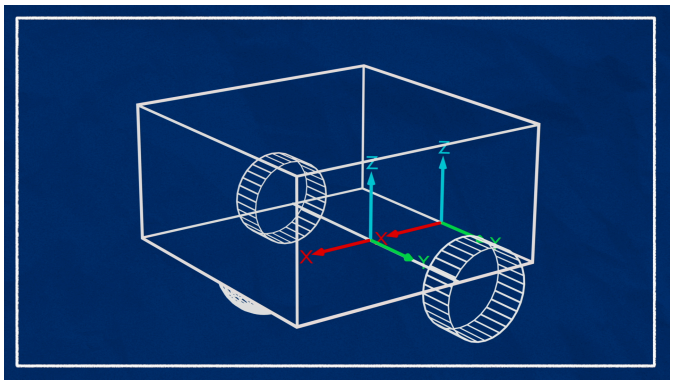
If you want you can relaunch robot state publisher now and check everything runs (make sure to rebuild the workspace since we added a file).

Chassis

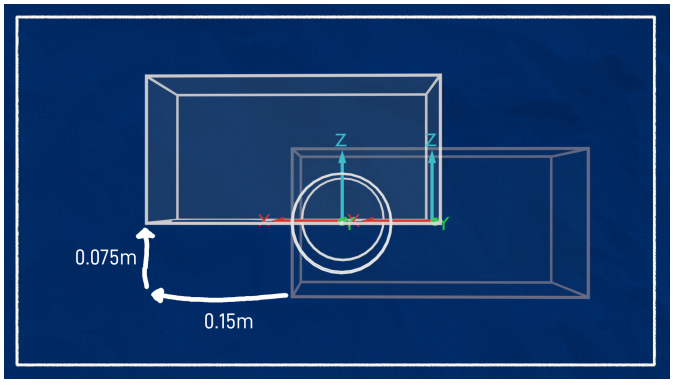
Next up is our chassis. Remember, we're not so interested in the final design right now, just the rough structure, so let's make it a box that is 300 × 300 × 150mm (for our American friends, this is about 1 × 1 × 1/2 ft). URDF values are in metres, so that's 0.3 × 0.3 × 0.15m.



I want the origin (the reference point) of the chassis to be at the bottom-rear-centre. So that means this `chassis` link will be connected to our `base_link` via a `fixed` joint, set back a little from the centre.



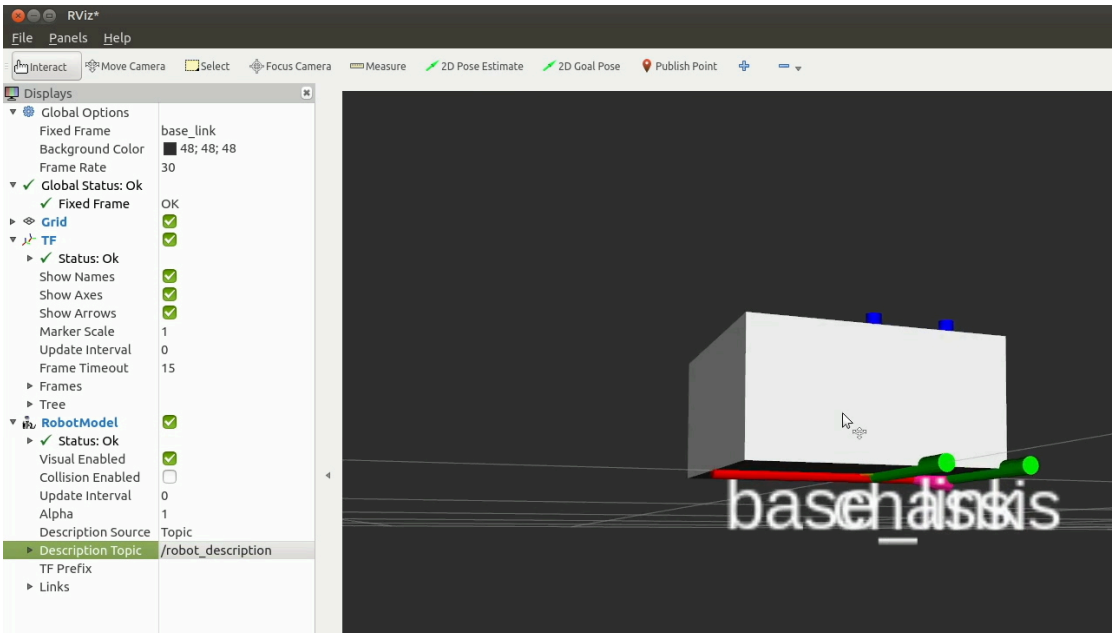
One other thing to note here is that by default the box geometry will be centred around the link origin. Recall that we want the link origin at the rear-bottom of the box, so to achieve that we want to shift the box forward (in X) by half its length (0.15m), and up (in Z) by half its height (0.075m).




```
<joint name="chassis_joint" type="fixed">
    <parent link="base_link"/>
```

```
    <child link="chassis"/>
    <origin xyz="-0.1 0 0"/>
</joint>

<link name="chassis">
    <visual>
        <origin xyz="0.15 0 0.075" rpy="0 0 0"/>
        <geometry>
            <box size="0.3 0.3 0.15"/>
        </geometry>
        <material name="white"/>
    </visual>
</link>
```

At this point we can fire up RViz as described earlier to see the transforms and visuals displayed - it should look like the image below.



> Note: If we wanted to, we could skip the chassis step and just add our boxes to the base link, but there are two reasons to do it this way:

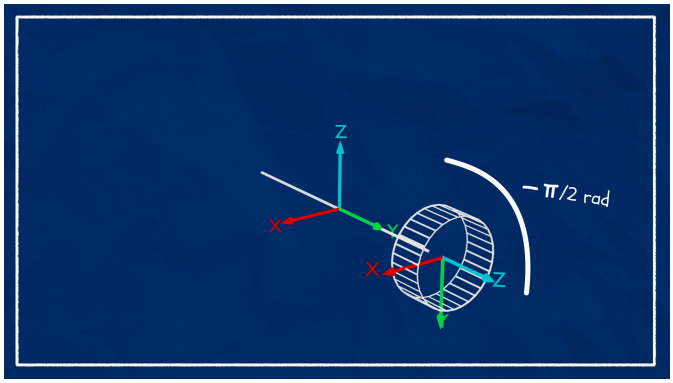
- `robot_state_publisher` will issue a warning if the root link has an inertia specified.
- If we have future things physically attached to the chassis (e.g. camera, lidar), by attaching them to a `chassis` link we can move the wheels and only change one number instead of all of them.

For our URDF to work properly we also need to add collision and inertia information, but we're going to get all the visuals sorted out first, and then come back through and add the other stuff.

Drive wheels

Now we want to add the drive wheels. The wheels can obviously move, so these will be connected to `base_link` via `continuous` joints. We could connect them to the chassis instead, but since we chose the base link to be at the centre of rotation it makes sense for the wheel links to be connected directly to it.

We want our wheels to be be cylinders oriented along the Y axis (left-to-right). In ROS though, cylinders by default are oriented along the Z axis (up and down). To fix this, we need to "roll" the cylinder by a quarter-turn around the X axis. I like to keep the Z-axis pointing outward (not inward), so I will rotate the left wheel clockwise (negative) around X by a quarter-turn ($-\frac{\pi}{2}$ radians), and the right wheel anticlockwise ($+\frac{\pi}{2}$ radians).



The last item of interest is the rotation `axis`. Since our left wheel has Z facing out, a "forward" drive would be an anticlockwise (positive) rotation around the Z axis. So we want the left wheel's axis to be +1 in Z.

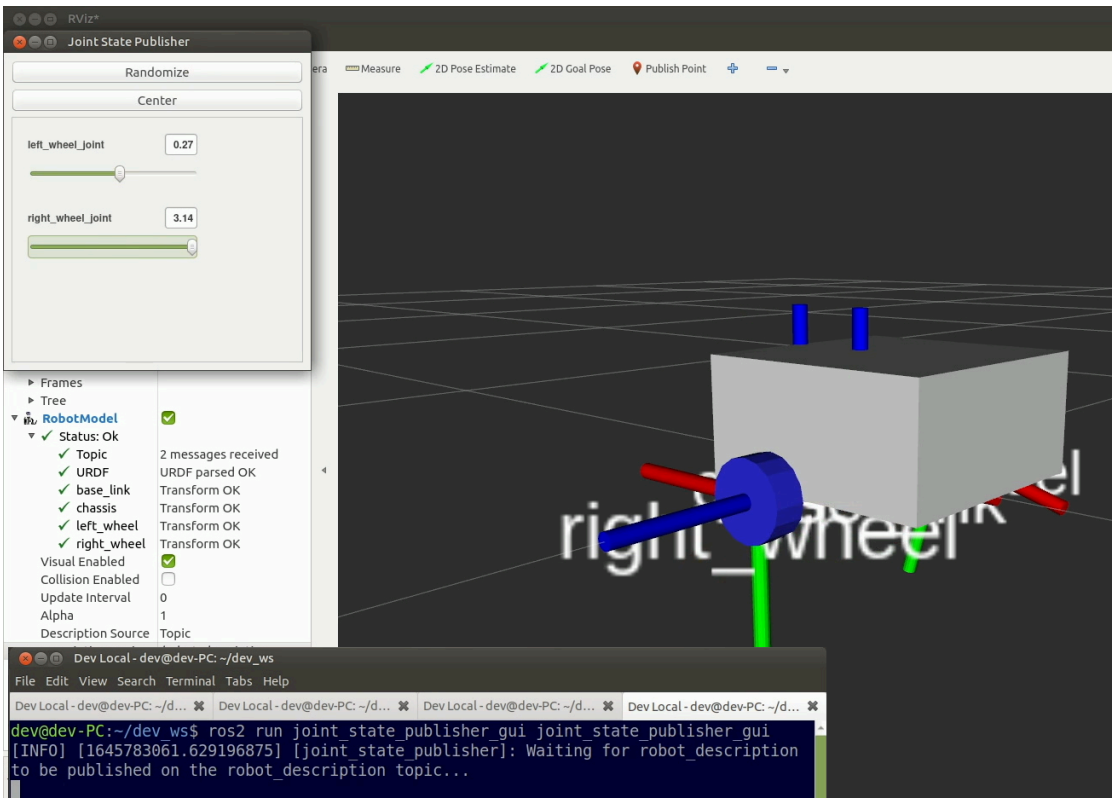
The full blocks for both wheels (including some chosen values for the cylinder length, radius, and Y offset - how wide the wheels are spaced apart from the centre) are shown below.

```
<!-- LEFT WHEEL -->

<joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
```

```
    <child link="left_wheel"/>
    <origin xyz="0 0.175 0" rpy="-${pi/2} 0 0"/>
    <axis xyz="0 0 1"/>
</joint>

<link name="left_wheel">
    <visual>
        <geometry>
            <cylinder length="0.04" radius="0.05" />
        </geometry>
        <material name="blue"/>
    </visual>
</link>
```

```
<!-- RIGHT WHEEL -->

<joint name="right_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="right_wheel"/>
    <origin xyz="0 -0.175 0" rpy="${pi/2} 0 0"/>
    <axis xyz="0 0 -1"/>
</joint>

<link name="right_wheel">
    <visual>
        <geometry>
            <cylinder length="0.04" radius="0.05" />
        </geometry>
        <material name="blue"/>
    </visual>
</link>
```

If we try to view this in RViz now we'll notice that the wheels aren't displayed correctly, since nothing is publishing their joint states. We can temporarily run `joint_state_publisher_gui` to resolve this and see the wheels visualised.

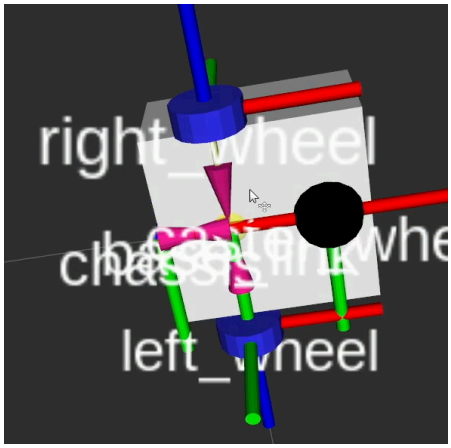


Caster wheel

Finally we need to add a caster wheel/s. The easiest way to do this is to simply add a frictionless sphere, connected to our chassis with the lowest point matching the base of the wheels. This isn't the most realistic physics simulation but is simpler to set up. Depending on the design, we sometimes want multiple caster wheels, but one will do for now. We'll cover the friction side of things in the next post.

```
<!-- CASTER WHEEL -->

<joint name="caster_wheel_joint" type="fixed">
    <parent link="chassis"/>
    <child link="caster_wheel"/>
    <origin xyz="0.24 0 0" rpy="0 0 0"/>
</joint>

<link name="caster_wheel">
    <visual>
        <geometry>
            <sphere radius="0.05" />
        </geometry>
        <material name="black"/>
```

```
    </visual>
</link>
```



Adding Collision and Inertia

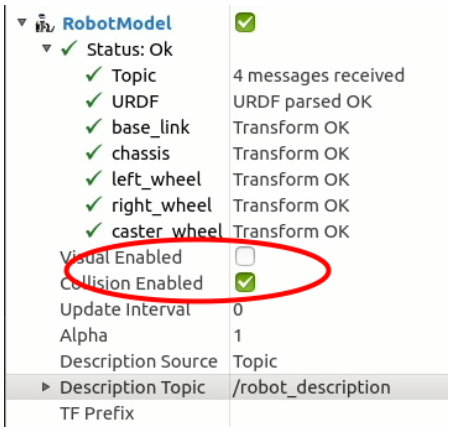
Adding collision

Once we've got our core visual structure tweaked the way we want it, we need to add collision. The simplest way to do this is to copy-and-paste the `geometry` and `origin` from our `<visual>` tags into `<collision>` tags.

For example:

```
<link name="chassis">
    <visual>
        <origin xyz="0.15 0 0.075" rpy="0 0 0"/>
        <geometry>
            <box size="0.3 0.3 0.15"/>
        </geometry>
        <material name="white"/>
    </visual>
    <collision>
        <origin xyz="0.15 0 0.075" rpy="0 0 0"/>
        <geometry>
            <box size="0.3 0.3 0.15"/>
        </geometry>
    </collision>
</link>
```

So go ahead and do this for all the links. To check if the collision geometry looks correct, in the RViz RobotModel display, we can untick "Visual Enabled" and tick "Collision Enabled" to see the collision geometry (it will use the colours from the visual geometry).



It's worth noting that this method isn't really ideal - if we need to change any parameters (e.g. our wheels have a larger radius) we now have even more places to change them! Instead I strongly recommend you go through and use `xacro` properties to define your structure.

Adding inertia

The last thing we need to add are our `<inertia>` tags. As noted in the URDF tutorial, calculating inertia values can sometimes be tricky, and it's often easier to use macros. Download the `inertia_macros.xacro` file from here and place it in your `description/` directory. Then, near the top of your `robot_core.xacro` file (or `robot.urdf.xacro`) just under the opening `<robot>` tag, add the following line to include them.

```
<xacro:include filename="inertial_macros.xacro" />
```

Once that's in, go ahead and add the relevant macro to each link. Note that you'll need to specify an `<origin>` tag when using these macros. This should match the visual/collision origin, NOT the joint origin. So if the visual/inertial didn't have an origin, just specify it as all zeros.

Here are my examples:

```
<link name="chassis">
     <!-- ... visual and collision here ... -->
    <xacro:inertial_box mass="0.5" x="0.3" y="0.3" z="0.15">
        <origin xyz="0.15 0 0.075" rpy="0 0 0"/>
    </xacro:inertial_box>
</link>

<link name="left_wheel">
    <!-- ... visual and collision here ... -->
    <xacro:inertial_cylinder mass="0.1" length="0.05" radius="0.05">
        <origin xyz="0 0 0" rpy="0 0 0"/>
    </xacro:inertial_cylinder>
</link>

<!-- Right wheel is same as left for inertia -->

<link name="caster_wheel">
    <!-- ... visual and collision here ... -->
    <xacro:inertial_sphere mass="0.1" radius="0.05">
        <origin xyz="0 0 0" rpy="0 0 0"/>
    </xacro:inertial_sphere>
</link>
```

Extending on this

We now have a basic structure laid out that we can expand upon as we work on our design. As we add new aspects (sensors etc.) we can just keep creating and including more `xacro` files! This makes for a nicely modular, flexible, structured system.

To view the full project as it stands at this point, click here.

The next step is to spawn our robot in Gazebo and drive it around with the keyboard. This post is long enough as it is, so the next part is in a separate post

9 replies

gdp-si Mar '23

vboxuser@linux1:~/dev_wc$ ros2 launch my_bot rsp.launch.py
ros2: command not found
i get the above error in the initial step, please help!!

After soutcing ros i got below error
Package 'my_bot' not found: "package 'my_bot' not found, searching: ['/opt/ros/foxy']"

1 reply

JoshNewans ▶ gdp-si Mar '23

It looks like you've either not built your workspace (with `colcon`) or not sourced it (with `source install/setup.bash`).

When you source the main ROS install (with `source /opt/ros/foxy/setup.bash`) that makes all the installed packages available but not ones you've built yourself. For those you need to source your local workspace (sometimes called an *overlay*) with `source install/setup.bash` (called from within the workspace) to make its packages available. It can get a bit confusing but once you are used to it it's actually a very practical approach for working on multiple projects.

bluehaven Jul '23

How would I prepare a diff robot with a caster at the back? I prepared one but the physical left wheel is shown on the right side in Rviz. How do I fix it? @JoshNewans

```
84% of storage used … You can clean up space or get more storage for Drive, Gmail, and Google Photos.
<?xml version='1.0'?>
<robot name="dd_robot">

  <!-- Base Link -->
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <box size="0.6096 0.4191 0.07"/>
      </geometry>
      <material name="blue">
        <color rgba="0 0.5 1 1"/>
      </material>
    </visual>
    <!-- Base collision, mass and inertia -->
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <box size="0.6096 0.4191 0.07"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="5"/>
      <inertia ixx="0.13" ixy="0.0" ixz="0.0" iyy="0.21" iyz="0.0" izz="0.13"/>
    </inertial>

    <!-- Caster -->
    <visual name="caster">
      <origin xyz="-0.3 0.0 -0.07" rpy="0 0 0" />
      <geometry>
        <sphere radius="0.0381" />
      </geometry>
    </visual>
    <!-- Caster collision, mass and inertia -->
    <collision>
      <origin xyz="-0.3 0.0 -0.07" rpy="0 0 0" />
      <geometry>
        <sphere radius="0.00381" />
      </geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.0001" ixy="0.0" ixz="0.0" iyy="0.0001" iyz="0.0" izz="0.0001"/>
    </inertial>

  </link>

  <!-- Right Wheel -->
  <link name="right_wheel">
    <visual>
      <origin xyz="0 0 0"  rpy="1.570795 0 0" />
      <geometry>
        <cylinder radius="0.0762" length="0.04318"/>
      </geometry>
      <material name="darkgray">
        <color rgba=".2 .2 .2 1"/>
      </material>
    </visual>
    <!-- Right Wheel collision, mass and inertia -->
    <collision>
      <origin xyz="0 0 0"  rpy="1.570795 0 0" />
      <geometry>
        <cylinder radius="0.0762" length="0.04318"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0" iyy="0.005" iyz="0.0" izz="0.005"/>
    </inertial>

  </link>

  <!-- Right Wheel joint -->
  <joint name="joint_right_wheel" type="continuous">
    <parent link="base_link"/>
    <child link="right_wheel"/>
    <origin xyz="0.2 0.24 0"  rpy="0 0 0" />
    <axis xyz="0 1 0" />
  </joint>

  <!-- Left Wheel -->
  <link name="left_wheel">
    <visual>
      <origin xyz="0 0 0"  rpy="1.570795 0 0" />
```

```
      <geometry>
        <cylinder radius="0.0762" length="0.04318"/>
      </geometry>
      <material name="darkgray">
        <color rgba=".2 .2 .2 1"/>
      </material>
    </visual>
    <!-- Left Wheel collision, mass and inertia -->
    <collision>
      <origin xyz="0 0 0" rpy="1.570795 0 0" />
      <geometry>
        <cylinder radius="0.0762" length="0.04318"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.01" ixy="0.0" ixz="0.0" iyy="0.005" iyz="0.0" izz="0.005"/>
    </inertial>
  </link>

  <!-- Left Wheel joint -->
  <joint name="joint_left_wheel" type="continuous">
    <parent link="base_link"/>
    <child link="left_wheel"/>
    <origin xyz="0.2 -0.24 0"  rpy="0 0 0" />
    <axis xyz="0 1 0" />
  </joint>

  <link name="caster">
    <inertial>
            <mass value="1" />
            <inertia ixx="0.1316" ixy="0.0" ixz="0" iyy="0.1316" iyz="0" izz="0.01125" />
    </inertial>
    <visual>
      <geometry>
        <sphere radius="0.0381" />
      </geometry>
      <material name="white" />
        <color rgba="1 1 1 1"/>
    </visual>
  </link>
  <joint name="caster_joint" type="continuous">
    <origin xyz="-0.3 0.0 -0.07" rpy="1.57 0.0 0.0"/>
    <parent link="base_link"/>
    <child link="caster"/>
  </joint>

  <link name="laser">
    <inertial>
            <mass value="1.0" />
            <inertia ixx="0.004375" ixy="0.0" ixz="0" iyy="0.004375" iyz="0" izz="0.005" />
    </inertial>
    <visual>
      <geometry>
        <cylinder radius="0.05" length="0.05"/>
      </geometry>
      <material name="grass">
        <color rgba="0.3607843137254902 0.6745098039215687 0.17647058823529413 1.0" />
      </material>
    </visual>
  </link>
  <joint name="laser_joint" type="fixed">
    <origin xyz="0.225 0 0.07" rpy="0 0 3.1416"/>
    <parent link="base_link"/>
    <child link="laser"/>
    <axis xyz="0.0 0.0 1.0"/>
  </joint>

</robot>
```

dhudson82 Mar '24

Josh,
Like everyone, I exceptionally greatful for the effort you've put into these tutorials.
I rushed through the first build out and then updated for humble. I may have something out of alignment as I restarted from the beginning on the tuturials. I have an issue my bot articubot_one2 builds fine, displays in rivz2 fine, but I can't get it to show in Gazebo. I see my spawn in Gazebo, but there are no links associated with it. I don't appear to get any errors on the spawn command. I do see a durability.policy warning, but it was suggested that wasn't significant. Just wanted to ping you for any ideas? My goal is to complete this robot, then add gps and then build a vineyard model to map and explore a vineyard in preparation for automation.

---

lokesh Mar '24

error: "No tf data. Actual error. Frame [map] does not exist."
I'm following this tutorial… in this video when I launch the rviz2 and while fixing the fixed frame to the base_link
There is the error I face …I cant find the base_link in the dropdown menu. The drop-down menu has nothing
if I manually type it as base_link then it shows "No tf data. Actual error. Frame [base_link] does not exist."

can someone help me with this error?

thanks in advance!

2 replies

Mohd_Ibrahim Jun '24

Hi I saw your video "Why I think you should build this Robot". I am very interested in it and have been following through. The first video was good and smooth. In the second I have started getting a weird error. After that "EXAMPLE of base link " where u get that (segment base link) Output in terminal when u launch ROS2 run . launch.py . So when I do it it says robot initiliased. Please can you help in this matter I am ready to send screenshots. Your videos are very great. Thanks @JoshN

1 reply

---

umer ▶ lokesh Aug '24

i am having the same error could you help me if you have resolved it.

M.m ▶ lokesh 12 Jan

I have the same problem

3586 loknath ▶ Mohd_Ibrahim 16 Feb

3 **5300-loknath** ▶ Mohd_Ibrahim 16 Feb

after editing the urdf and running the commad ros2 launch my_bot rsp.launch.py
u will get something like got base_link got segment chassis
now open a new terminal and open the workspace and do the source install/setup.bash
and open rviz2
u will get the drop down menu with base_link and segment chassis